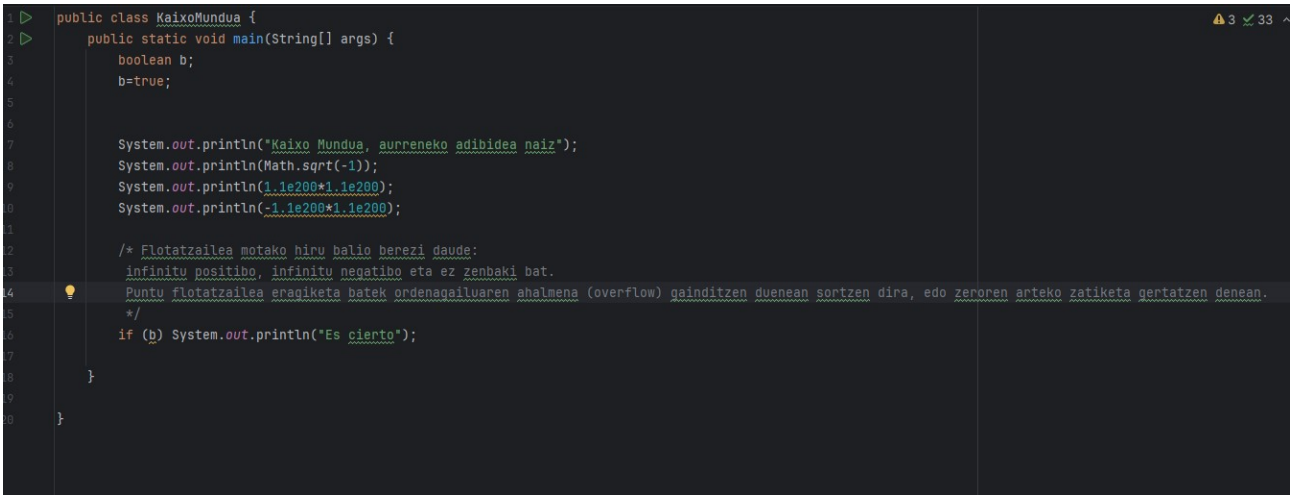
JAVA ERREPASOA

1 Ariketa (Kaixo Mundua)

- Javako programak bi kontzeptu ditu, puntu flotatzailea eragiketak, ezinezkoak edo oso handiak direnak (NaN (ez da zenbaki bat) Infinity eta – Infinity) eta beste aldetikan boolean balio bat (true/false) baldintza-egitura erabiltzen du (if).
- B aldagaian true ordez d false jartzen baduzue ez da es cierto mezua aterako
- If-ek egiaztatzen du egia den edo gezurra den eta horren arabera mezua agertuko da edo ez



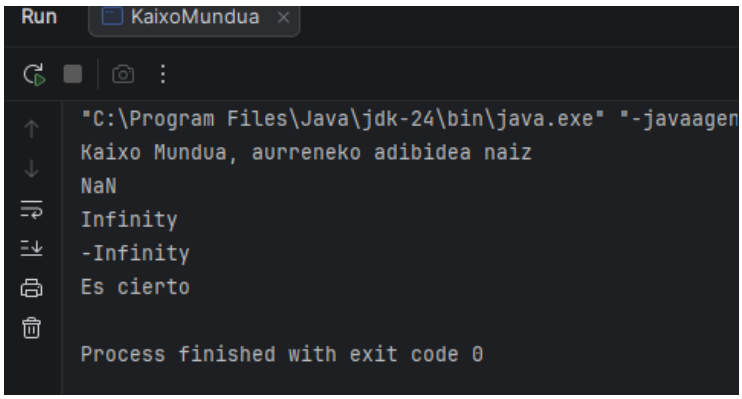
```java
public class KaixoMundua {
    public static void main(String[] args) {
        boolean b;
        b=true;


        System.out.println("Kaixo Mundua, aurreneko adibidea naiz");
        System.out.println(Math.sqrt(-1));
        System.out.println(1.1e200*1.1e200);
        System.out.println(-1.1e200*1.1e200);

        /* Flotatzailea motako hiru balio berezi daude:
         infinitu positibo, infinitu negatibo eta ez zenbaki bat.
         Puntu flotatzailea eragiketa batek ordenagailuaren ahalmena (overflow) gainditzen duenean sortzen dira, edo zeroren arteko zatiketa gertatzen denean.
         */
        if (b) System.out.println("Es cierto");

    }

}
```



```
Run   KaixoMundua

"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagen
Kaixo Mundua, aurreneko adibidea naiz
NaN
Infinity
-Infinity
Es cierto

Process finished with exit code 0
```

2 Ariketa (Bihurketak)

- Inplizitua: Datu txikiak handigoetan gordetzen dira
- Esplizitua: Mota handigoak tzikiagotzeko (mota) erabili behar da. Honek datuak mozeta (hamartarrak galtzea) edo gainezkatzea (overflow) ekartzeko
- Bateragarritasuna: Zenbakiak eta char beren artean bateragarriak dira, boolean berriz ez.

```java
public class Bihurketak{
/* Tamaina txikieneko datua automatikoki bihurtzen da tamaina handieneko mota. Cast operadorea behar da tipo handiago batetik txikiago baterako bihurketa egite
 Sei zenbaki-motak eta char mota bateragarriak dira bihurketetarako. Boolean mota bateraezina da gainerako guztiekin
 */


public static void main(String[] args) {
      char c='a'; //2 bytes
      int i2; //4 bytes
      short s; //2 bytes
      double d; //8 bytes
      float f; //4 bytes
      long l; // 8 bytes


      i2=c;
      System.out.println("i2= "+i2);
      //s=c; No lo permite
      s= (short) c;
      System.out.println("s= "+s);
      d=c;
      System.out.println("d= "+d);
      s=678;

      double dou=123.67;
      //int dest = dou; No lo permite
      int dest= (int) dou;
      System.out.println("dest= "+dest);

      dou=3.4028237E+50;
      f=(float) dou;
      System.out.println("f= "+f);
      dou=7.5;
      l=(long)dou;
      System.out.println("l= "+l);
```

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Prog
i2= 97
s= 97
d= 97.0
dest= 123
f= Infinity
l= 7
b= 97
b= 1

Process finished with exit code 0
```

```java
      int in=257;
      byte b; //1 byte
      b=(byte)c;
      System.out.println("b= "+b);
      b= (byte) in;
      System.out.println("b= "+b);

    }
}
```

3 Ariketa (Bilgarriak)

- Zenbaki bakoitzak bere oinarrizko datu-mota dauke, adibidez 124 osoa danez int, gero daude beste batzuk adibidez 3,1, hamartarra/float da ordun inte ezin du izan ze oi osoatan bakarrik erabiltzen da, jartzen baduzu errorea emangu du, gero de zenbaki handi bat egoten bada, 165000 adibidez, long mota izango zan, int motaren muga gainditzen dute zenbakiak, laburbiduz, bakoitzak bere datu mota

```java
/* Bilgarri klaseak (Wrappers): oinarrizko tipo bat objektu bihurtzea ahalbidetzen dute.
9 daude: Integer, Long, Float, Double, Short, Byte, Character, Boolean eta Void. Karaktere-kateak oinarrizko mota bihurtzeko aukera ematen duten metodoak erabil da
 */

public class Bilgarriak {
public static void main(String[] args) {
System.out.println(Integer.parseInt( s: "124"));
System.out.println(Long.parseLong( s: "1650097"));
System.out.println(Byte.parseByte( s: "21"));
System.out.println(Short.parseShort( s: "45"));
System.out.println(Float.parseFloat( s: "124.89"));
System.out.println(Double.parseDouble( s: "1.5e8"));
// Biderketa, mota egokiarekin egin ahal izateko
System.out.println(Double.parseDouble( s: "3.1416")*2);

}

}
```

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "
124
1650097
21
45
124.89 Explain with EasyCode
1.5E8 Explain with EasyCode
6.2832 Explain with EasyCode

Process finished with exit code 0
```

4 Ariketa (Aldagaiak)

- A aldagiak int 9 balio du int balioarekin, b balioa ateratzeko jarri du, a balioa + 1
- Gero pantailan aterakoa lehenengo b balio eta gero a balio horrela dagoelako jarrrita
- `C` (Kaixo) eta `D` (Luis) aldagaiak sortzen dira. Ondoren, `e` aldagai berria sortzen da biak lotzeko (`C + D`), eta pantailan `"Kaixo Luis"` inprimatzen da.

```java
/* Aldagai bat definitzen den kode-blokeak zehazten du haren eremua
 */

package Aldagaiak;
public class Aldagaiak {
    public static void main(String[] args) {
        int a;
        a=9;
        {
            int b=a+1;
            System.out.println("b: " +b);
        }
        //System.out.println("b: "+b);
        System.out.println("a: "+a);
        String c,d;
        c="Kaixo";
        d=" Luis";
        String e= c+d;
        System.out.println(e);

    }
}
```

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-j
b: 10
a: 9
Kaixo Luis

Process finished with exit code 0
```

5 Ariketa (Dimentsio bakarreko array-ak)

- Array aint motako array edo bektore bat sortzen da, zenbaki osoak gordetzeko. Tamaina 3 da (new int[3]), beraz hiru posizio ditu: 0, 1 eta 2. Posizio bakoitza gordetzen da (3, 6 eta 9)
- Println bidez posizio bakoitzeko balioa inprimatzen da. Azkenik, manzanas.length erabiltzen da array-ak guztira zenbat elementu dituen erakusteko (3).

```java
/* Javan array guztiak objektuak
*/
package Dimentsioa;
public class Dimentsioa {
    public static void main(String[] args) {
        int manzanas[];
        manzanas=new int[3];
        manzanas[0]=3;
        manzanas[1]=6;
        manzanas[2]=9;

        System.out.println("Compré "+ manzanas[0]+ " manzanas el lunes");
        System.out.println("Compré "+ manzanas[1]+ " manzanas el martes");
        System.out.println("Compré "+ manzanas[2]+ " manzanas el miércoles");
        System.out.println("Días de la semana "+ manzanas.length);

    }
}
```

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-java
Compré 3 manzanas el lunes
Compré 6 manzanas el martes
Compré 9 manzanas el miércoles
Días de la semana 3

Process finished with exit code 0
```

6 Ariketa (Array bidimentsionalak)

- Bi dimentsiotako array bat sortzen da (`int[][]`). Lehenengo dimentsioa bakarrik zehaztu egin behar da, 3 errenkada daude
- Errenkada bakoitzak kopuru bat du, 0 erremkada 1, 1 errenkada 2 eta 2 errenkada 3
- tabla.length-ek errenkada kopurua ematen du, eta tabla[i].length-ek errenkada bakoitzeko zutabe kopurua

```java
/* Objektu array non elementu bakoitza beste objektu array bat den */
package ArrayBidimentsionala;
public class ArrayBidimentsionala {
    public static void main(String[] args) {
        //Lehenengo dimentsioa bakarrik da nahitaezkoa

        int [][] tabla =new  int [3][];

        tabla[0]=new int [1];
        tabla[1]=new int [2];
        tabla[2]=new int [3];

        System.out.println("Errenkada kopurua: "+tabla.length);
        System.out.println("Errenkada 0. Zutabeak: "+tabla[0].length);
        System.out.println("Errenkada 1. Zutabeak: "+tabla[1].length);
        System.out.println("Errenkada 2. Zutabeak: "+tabla[2].length);

        //Posible da adierazpenaren unean array bat hasieratzea:
        //int [][] tabla ={{0}, {0,0}, {0,0,0}};

    }
}
```

```
C:\Program Files\Java\jdk-24\bin\java.exe
Errenkada kopurua: 3
Errenkada 0. Zutabeak: 1
Errenkada 1. Zutabeak: 2
Errenkada 2. Zutabeak: 3

Process finished with exit code 0
```

7 Ariketa (Metodo Estatikoak)

- String motako zerrenda bat sortu asteko egunekin eta alda gai bat gaurko egunarekin edo egun batekin
- Metodoak static hitza duenez, ez da objekturik sortu behar
- for begiztak gaur zein posiziotan dagoen bilatzen du. Azken eguna denez (astea.length - 1), hurrengoa = 0 egiten du berriro hasierara bueltatzeko
- Metodoak astelehena itzultzen du eta pantailan inprimatzen da, ze aldagaien igandea jartzen du

```java
/* Metodo estatikoa edo klase-metodoa erabiltzea. Ez da objektu bat sortu behar hori eskatzeko */

package MetodoEstatikoak;
public class MetodoEstatikoak {
    public static void main(String[] args) {
        String astea[]={"astelehena", "asteartea", "asteazkena", "osteguna", "ostirala","larunbata", "igandea"};
        String gaur="igandea";
        String hurrengoa=hurrengoEguna(astea,gaur);
        System.out.println(gaur+"-ren hurrengo eguna hau da: "+hurrengoa);
    }


    public static String hurrengoEguna(String[] astea, String eguna){ 1 usage
        int posizioa=0, hurrengoa;
        for (int i=0; i<astea.length;i++)
            if (astea[i].equals(eguna))
                posizioa=i;
        if (posizioa ==astea.length-1)
            hurrengoa =0;
        else
            hurrengoa=posizioa +1;
        return astea[hurrengoa];
    }
```

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Program Files\J
igandea-ren hurrengo eguna hau da: astelehena

Process finished with exit code 0
```

8 Ariketa (Instantzia metodoak edo ez estatikoak)

- String baten, asteko egunak jartzen ditu, string erabiltzen du hitzetako erabiltzen delako, zenbaki osok egoten baziren adbidez int erabilio zen
- Beste string bat jartzen du, aldagai bat gaurko egunakin edo beste batekin
- for begiztak gaurko eguna zerrendan bilatzen du eta bere posizioa topatzen du. Ondoren, hurrengo posizioko eguna aukeratzen du (+1). Eguna azkena bada (domingo), lehenengorako bidea hartzen du berriro (lunes)
- return bidez hurrengo eguna itzultzen du, eta System.out.println-ek pantailatik inprimatzen du.

```java
/* Metodo EZ estatikoa erabiltzea. Objektu bat sortu behar da hori eskatzeko */

package MetodoEzEstatikoa;
public class MetodoEzEstatikoa {
    public static void main(String[] args) {
        String semana[]={"lunes", "martes", "miercoles", "jueves", "viernes","sabado", "domingo"};
        String hoy="domingo";
        MetodoEzEstatikoa mez=new MetodoEzEstatikoa();
        String siguiente=mez.diaSiguiente(semana,hoy);
        System.out.println("El día siguiente a "+hoy+ " es: "+siguiente);
    }

    public String diaSiguiente(String[] semana, String dia){ 1 usage
        int posicion=0, siguiente;
        for (int i=0; i<semana.length;i++)
            if (semana[i].equals(dia))
                posicion=i;
        if (posicion ==semana.length-1)
            siguiente =0;
        else
            siguiente=posicion +1;
        return semana[siguiente];
    }
```

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaager
El día siguiente a domingo es: lunes

Process finished with exit code 0
```

9 Ariketa (Parametroak)

- Hasieran aldagaiak balio bat du, gero metodoa erabiliza balio aldatu egiten da baino bakarrik kopia bat egiten du, horregatik kanpoko a aldagaia 0 izaten segitzen du
- int v[] array-a metodoari pasatzean, memoriako leku berdinera bideratzen duen erreferentzia bat pasatzen da. Metodo barruan v[0] = 7 egitean, jatorrizko array-a aldatu egiten da.

```java
/* Oinarrizko motak baliotzat (kopiak) eta objektuak erreferentziatzat hartzen dira */

package Parametroak;
public class Parametroak {
    public static void main(String[] args) {
        int a=0;
        int v[]={0};
        System.out.println("antes: a="+a+", v[0]="+v[0]);
        valorEntero(a, valor: 7);
        valorVector(v, valor: 7);
        System.out.println("despues: a="+a+", v[0]="+v[0]);
    }

    public static void valorEntero(int a, int valor){  1 usage
        //variable a pasa por valor
        a=valor;
    }
    public static void valorVector(int[] v, int valor){  1 usage
        //variable v pasa por referencia
        v[0]=valor;
    }
```

```
"C:\Program Files\Java\jdk-24\bin\jav
antes: a=0, v[0]=0
despues: a=0, v[0]=7

Process finished with exit code 0
```

10 Ariketa (Salbuespenak)

- Akats bat sortu dezakeen kodea jartzen da barruan, kasu honetan, 10/0 egitea, zeroz zatitzea ezinezkoa delako. Akatsa gertatzen den unean, bloke horretako exekuzioa moztu egiten da
- try barruan akats bat gertatzen denean, adibidez, ArithmeticException, catch blokeak akats hori harrapatu eta kudeatzen du, programa bat-batean oker gelditu beharrean
- Akatsa kudeatu denean, programak exekutatzen jarraitzen du catch bloketik behera dagoen guztia inprimatuz.

```java
/* Errore aritmetikoaren kudeaketa, 0ren arteko zatiketa, programa ustekabean ez amaitze
*/

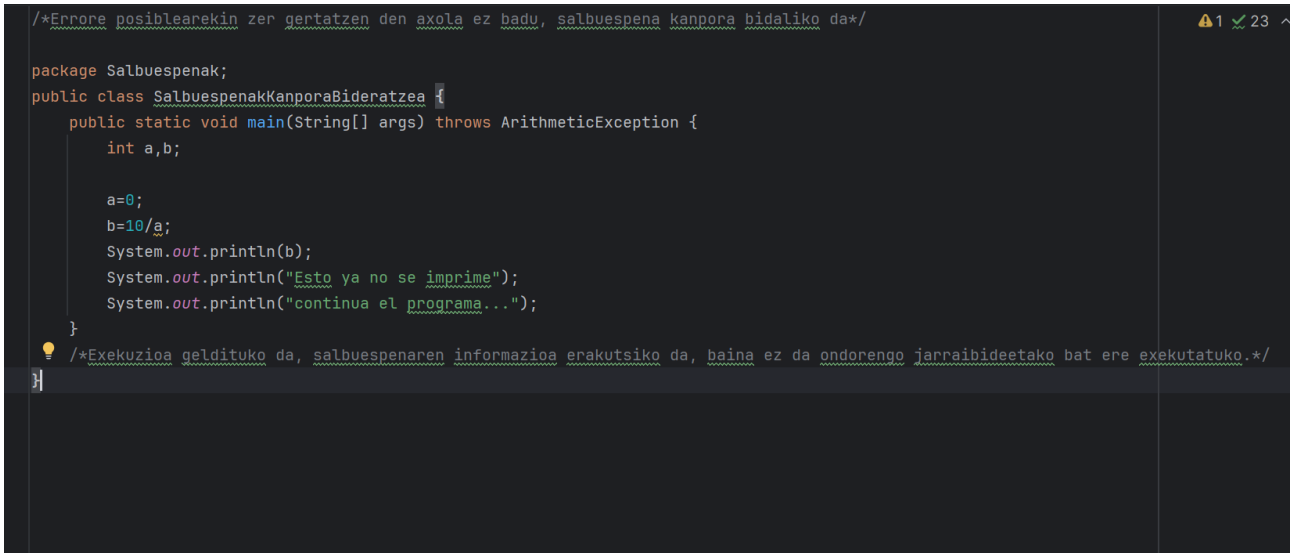
package Salbuespenak;
public class Salbuespenak {
public static void main(String[] args) {
    int a,b;
    try{
        a=0;
        b=10/a;
        System.out.println("Esto ya no se imprime");
    }catch (ArithmeticException objetoExcep){
        System.out.println("División por cero "+ objetoExcep);
    }
    System.out.println("continua el programa...");
    }
/*    try{
  *       bloque de código peligroso
  *    }
  *    catch (TipoExcepción1 ex0b){
  *      gestión de excepciones pata TipoExcepción1
```

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Program
División por cero java.lang.ArithmeticException: / by zero
continua el programa...

Process finished with exit code 0
```

11 Ariketa (Salbuespenak kanpora bideratzea)

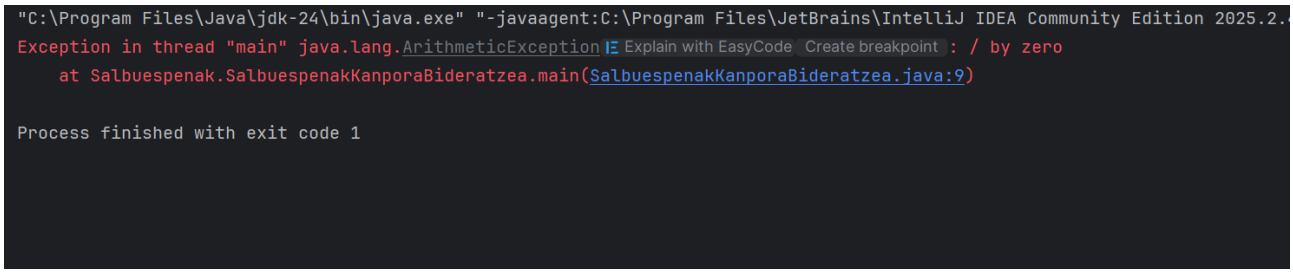
- Metodoan throws ArithmeticException jartzean, programak esaten du metodo honek salbuespena sortzen duela baina ez duela barruan kudeatuko try-catch ez dagoelako
- Zeroz zatitzea gertatzean 10 / a, a aldagaia 0 da , salbuespena bat agertzen dax eta programa berehala gelditzen da.
- Zatiketaren ondorengo lerroak println ez dira exekutatzen, ezta programaren amaierakoak ere. Konsolan emaitza erroreen pilaren informazioa erakutsiko da



```
/*Errore posiblearekin zer gertatzen den axola ez badu, salbuespena kanpora bidaliko da*/

package Salbuespenak;
public class SalbuespenakKanporaBideratzea {
    public static void main(String[] args) throws ArithmeticException {
        int a,b;

        a=0;
        b=10/a;
        System.out.println(b);
        System.out.println("Esto ya no se imprime");
        System.out.println("continua el programa...");
    }
    /*Exekuzioa geldituko da, salbuespenaren informazioa erakutsiko da, baina ez da ondorengo jarraibideetako bat ere exekutatuko.*/
}
```



```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.2.
Exception in thread "main" java.lang.ArithmeticException Explain with EasyCode  Create breakpoint : / by zero
    at Salbuespenak.SalbuespenakKanporaBideratzea.main(SalbuespenakKanporaBideratzea.java:9)

Process finished with exit code 1
```

12 Ariketa (Datuak Jaso)

- Testu-kateak String irakurtzeko erabiltzen da adibidez, izena: cadena
- Zenbaki osoak int irakurtzeko adibidez, adina: adina
- Zenbaki hamartarrak double irakurtzeko adibidez, altuera/garaiera: altuera
- adina + altuera egiterakoan, Javan zenbaki oso bat int eta hamartar bat double gehitzean, emaitza automatikoki zenbaki hamartarra double bihurtzen da.

```java
package DatuakJaso;
import java.util.*;
public class DatuakJaso {
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String cadena;
        int adina;
        double altuera;

        System.out.println("Zure izena idatzi: ");
        cadena = sc.nextLine();
        System.out.println("Izena duzu:  "+cadena);
        System.out.println("Zure adina idatzi: ");
        adina = sc.nextInt();
        System.out.println("Dauzkazu: "+adina +" urte");
        System.out.println("Idatzi zure garaiera hamartarrekin: ");
        altuera = sc.nextDouble();
        System.out.println("Neurria: "+altuera+" metro");
        System.out.println("Adina + altuera= "+ (adina+altuera));
    }
}
```

```
C:\Program Files\Java\jdk-24\bin\java.exe "-javaag
Zure izena idatzi:
Aimar
Izena duzu:  Aimar
Zure adina idatzi:
18
Dauzkazu: 18 urte
Idatzi zure garaiera hamartarrekin:
1,80
Neurria: 1.8 metro
Adina + altuera= 19.8

Process finished with exit code 0
```

13 Ariketa (ArrayList)

- Enkapsulazioa aplikatzen da private, datuak babesteko equipoLocal, equipoVisitante, golesLocal, golesVisitante.
- Objektu berri baten datuak hasieratzeko erabiltzen dira
- Balioak modu seguruan irakurtzeko get eta aldatzeko set metodoak. Get aldagai baten balioa irakurtzeko edo lortzeko eta Set aldagai baten balioa aldatzeko
- Partiduaren emaitza modu formatutuan inprimatzen du
- ArrayList<PartidoFutbol> listaPartidos = new ArrayList<>(); erabiliz, PartidoFutbol motako elementuak bakarrik gordeko dituen zerrenda dinamiko bat sortzen da.
- Bucle baten bidez elementu bakoitza irakurri eta bere mostrarInfo() metodoa exekutatzen da.
- equals() metodoa erabiltzen da testuak alderatzeko adibidez. etxeko taldea Real Madrid den egiaztatzeko
- listaPartidos.remove(0) erabiliz, 0 posizioko lehen partidua zerrendatik kentzen da.

```java
import java.util.ArrayList;

Rename usages
public class Partido {
    public static void main(String[] args) {
        // Crear un ArrayList de objetos PartidoFutbol
        ArrayList<PartidoFutbol> listaPartidos = new ArrayList<>();

        // Añadir partidos
        listaPartidos.add(new PartidoFutbol( local: "Real Madrid",  visitante: "Barcelona",  golesLocal: 2,  golesVisitante: 1));
        listaPartidos.add(new PartidoFutbol( local: "Atlético de Madrid",  visitante: "Sevilla",  golesLocal: 0,  golesVisitante: 0));

        // Mostrar información de los partidos
        for (PartidoFutbol partido : listaPartidos) {
            partido.mostrarInfo();
        }

        // Buscar un partido por equipo local
        String equipoBuscado = "Real Madrid";
        for (PartidoFutbol partido : listaPartidos) {
            if (partido.getEquipoLocal().equals(equipoBuscado)) {
                System.out.println("Encontrado: " + partido.getEquipoLocal());
            }
        }

        // Eliminar un partido (por índice o por objeto)
        listaPartidos.remove( index: 0); // Elimina el primer partido (Real Madrid vs. Barcelona)

        // Mostrar información actualizada
        System.out.println("Partidos restantes:");
        for (PartidoFutbol partido : listaPartidos) {
            partido.mostrarInfo();
        }
    }
}
```

```java
    private String equipoLocal;  4 usages
    private String equipoVisitante;  4 usages
    private int golesLocal;  4 usages
    private int golesVisitante;  4 usages

    // Eraikitzailea (Constructor)
    public PartidoFutbol(String local, String visitante, int golesLocal, int golesVisitante) {  2 usages
        this.equipoLocal = local;
        this.equipoVisitante = visitante;
        this.golesLocal = golesLocal;
        this.golesVisitante = golesVisitante;
    }

    // Getters
    public String getEquipoLocal() {  2 usages
        return equipoLocal;
    }

    public String getEquipoVisitante() {  no usages
        return equipoVisitante;
    }

    public int getGolesLocal() {  no usages
        return golesLocal;
    }

    public int getGolesVisitante() {  no usages
        return golesVisitante;
    }

    // Setters
    public void setEquipoLocal(String equipoLocal) {  no usages
        this.equipoLocal = equipoLocal;
    }

    public void setEquipoVisitante(String equipoVisitante) {  no usages
        this.equipoVisitante = equipoVisitante;
    }
```

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaage
Real Madrid vs. Barcelona | Resultado: 2-1
Atlético de Madrid vs. Sevilla | Resultado: 0-0
Encontrado: Real Madrid
Partidos restantes:
Atlético de Madrid vs. Sevilla | Resultado: 0-0

Process finished with exit code 0
```

```java
    }

    public void setGolesLocal(int golesLocal) {  no usages
        this.golesLocal = golesLocal;
    }

    public void setGolesVisitante(int golesVisitante) {  no usages
        this.golesVisitante = golesVisitante;
    }

    // Informazioa bistaratzeko metodoa
    public void mostrarInfo() {  2 usages
        System.out.printf("%s vs. %s | Resultado: %d-%d%n",
                equipoLocal, equipoVisitante, golesLocal, golesVisitante);
    }
}
```